Architecture Discovery Report

Date: 2025-07-14
Project Root: `C:\Users\davidmo\repos\logfire`

---

1. Project Overview

- **Project Type**: Open-source Python SDK / Library + CLI tool. Part of a larger SaaS observability platform (the backend/UI are closed source). Distributed as a PyPI package.
- **Languages**: Python 3.9–3.14 (primary); JavaScript/Node.js (minor — Pyodide/browser compatibility test harness only)
- **Frameworks / Core Protocols**:
  - OpenTelemetry SDK (`opentelemetry-sdk >= 1.39.0, < 1.40.0`) — traces, metrics, logs
  - OTLP HTTP exporter (`opentelemetry-exporter-otlp-proto-http`) — telemetry transport
  - OpenTelemetry Instrumentation (`opentelemetry-instrumentation`) — base instrumentation framework
  - Rich (`rich >= 13.4.2`) — console output formatting
  - Protobuf (`protobuf >= 4.23.4`) — OTLP serialization
  - Executing (`executing >= 2.0.1`) — source code / f-string magic inspection
- **Version**: 4.31.0
- **Build System**: Hatchling
- **Package Manager**: `uv` (with `uv.lock` lockfile committed)

- **Target Audience**: Python developers building applications who want distributed tracing, structured logging, and metrics — particularly those already using Pydantic, FastAPI, or other Python-centric stacks.
- **Monetization**: Logfire SaaS platform subscription (`logfire.pydantic.dev`); self-hosted enterprise license available. SDK itself is MIT-licensed and free.

---

2. Directory Structure

```
logfire/                        # Root of the repository (uv workspace)
│
├── logfire/                    # Main SDK Python package
│   ├── __init__.py             # Public API surface — all exported symbols
│   ├── __main__.py             # Enables `python -m logfire`
│   ├── cli.py                  # Thin re-export of _internal.cli.main
│   ├── testing.py              # Pytest fixtures and test utilities (CaptureLogfire, TestExporter, etc.)
│   ├── db_api.py               # PEP 249 DB API 2.0 wrapper over LogfireQueryClient
│   ├── query_client.py         # Re-export of experimental.query_client
│   ├── propagate.py            # OTel context propagation helpers
│   ├── types.py                # Shared public types (ExceptionCallback, etc.)
│   ├── exceptions.py           # LogfireConfigError and other public exceptions
│   ├── version.py              # VERSION constant (read from installed package metadata)
│   ├── py.typed                # PEP 561 marker — package ships type stubs
│   │
│   ├── sampling/               # Sampling API
│   │   ├── __init__.py         # SamplingOptions
│   │   └── _tail_sampling.py   # Tail-based sampling processor
│   │
│   ├── variables/              # Managed variables subsystem
│   │   ├── abstract.py         # VariableProvider ABC
│   │   ├── config.py           # VariablesConfig, VariableConfig types
│   │   ├── local.py            # Local (in-process) variable provider
│   │   ├── remote.py           # LogfireRemoteVariableProvider (HTTP polling)
```

```
│   │   └── variable.py         # Variable descriptor / accessor
│   │
│   ├── experimental/           # Experimental / unstable features
│   │   ├── query_client.py     # LogfireQueryClient / AsyncLogfireQueryClient (SQL over HTTP)
│   │   ├── api_client.py       # LogfireAPIClient (datasets & AI evals management)
│   │   ├── forwarding.py       # Proxy/forwarding OTLP export requests
│   │   └── annotations.py      # AI evaluation result annotation helpers
│   │
│   ├── integrations/           # Public-facing integration wrappers
│   │   ├── logging.py          # LogfireLoggingHandler (stdlib logging bridge)
│   │   ├── structlog.py        # LogfireProcessor (structlog bridge)
│   │   ├── loguru.py           # LogfireHandler (loguru bridge)
│   │   ├── aiohttp_client.py   # aiohttp client request/response hook types
│   │   ├── flask.py            # Flask-specific helpers
│   │   ├── httpx.py            # httpx-specific helpers
│   │   ├── psycopg.py          # psycopg-specific helpers
│   │   ├── redis.py            # Redis-specific helpers
│   │   ├── sqlalchemy.py       # SQLAlchemy-specific helpers
│   │   ├── wsgi.py             # WSGI-specific helpers
│   │   └── pydantic.py         # Pydantic plugin entry point
│   │
│   └── _internal/              # Private implementation (not part of public API)
│       ├── main.py             # Logfire class, LogfireSpan class — core tracing/logging API
│       ├── config.py           # configure(), LogfireConfig, all option dataclasses
│       ├── config_params.py    # Env var definitions, ParamManager, pyproject.toml config reading
│       ├── auth.py             # UserToken, UserTokenCollection, device-code auth flow
│       ├── client.py           # LogfireClient — HTTP client for Logfire management API
│       ├── tracer.py           # ProxyTracerProvider, PendingSpanProcessor
│       ├── metrics.py          # ProxyMeterProvider
│       ├── logs.py             # ProxyLoggerProvider
│       ├── formatter.py        # f-string / logfire_format magic
│       ├── scrubbing.py        # Sensitive data scrubbing (patterns + redaction)
│       ├── constants.py        # Span attribute key names, level numbers, etc.
│       ├── json_encoder.py     # Custom JSON serialization for span attributes
│       ├── json_schema.py      # JSON Schema generation for span attributes
```

```
│       ├── stack_info.py       # Source location extraction
│       ├── baggage.py          # OTel baggage utilities
│       ├── instrument.py       # @logfire.instrument() decorator implementation
│       ├── async_.py           # Async event-loop introspection / slow callback detection
│       ├── ulid.py             # ULID generation
│       ├── utils.py            # Shared utilities
│       ├── db_statement_summary.py  # SQL statement summarisation
│       ├── collect_system_info.py   # System info collection for spans
│       │
│       ├── auto_trace/         # AST-rewriting auto-trace subsystem
│       │   ├── __init__.py     # install_auto_tracing(), AutoTraceModule
│       │   ├── import_hook.py  # sys.meta_path import hook
│       │   ├── rewrite_ast.py  # AST transformer; @no_auto_trace decorator
│       │   └── types.py        # AutoTraceModule filter type
│       │
│       ├── cli/                # CLI command implementations
│       │   ├── __init__.py     # main() entry point, all subcommand parse_* handlers
│       │   ├── auth.py         # parse_auth, parse_logout
│       │   ├── prompt.py       # parse_prompt (AI agent prompt/issue lookup)
│       │   └── run.py          # parse_run, instrumentation context detection
│       │
│       ├── exporters/          # OTLP + other exporters
│       │   ├── otlp.py         # BodySizeCheckingOTLPSpanExporter, OTLPExporterHttpSession, DiskRetryer
│       │   ├── console.py      # Console span/log exporters (Rich-based)
│       │   ├── dynamic_batch.py  # DynamicBatchSpanProcessor
│       │   ├── test.py         # In-memory TestExporter, TestLogExporter
│       │   ├── logs.py         # Log processor wrappers
│       │   ├── processor_wrapper.py  # Suppress-instrumentation wrappers
│       │   ├── quiet_metrics.py  # Quiet (silent-on-error) metric exporter wrapper
│       │   ├── remove_pending.py  # RemovePendingSpansExporter
│       │   └── wrapper.py      # WrapperSpanExporter / WrapperLogExporter base classes
│       │
│       └── integrations/       # Framework instrumentation implementations (30+ files)
│           ├── fastapi.py      ├── django.py      ├── flask.py
│           ├── starlette.py    ├── asgi.py        ├── wsgi.py
```

```
│           ├── httpx.py        ├── requests.py    ├── aiohttp_client.py
│           ├── aiohttp_server.py
│           ├── sqlalchemy.py   ├── asyncpg.py     ├── psycopg.py
│           ├── sqlite3.py      ├── mysql.py       ├── pymongo.py
│           ├── redis.py        ├── celery.py      ├── aws_lambda.py
│           ├── system_metrics.py
│           ├── pydantic_ai.py  ├── openai_agents.py ├── claude_agent_sdk.py
│           ├── google_genai.py ├── litellm.py     ├── dspy.py
│           ├── mcp.py          ├── surrealdb.py
│           ├── print.py        ├── executors.py   ├── pytest.py
│           └── llm_providers/  # Shared LLM provider helpers
│               ├── openai.py   ├── anthropic.py   ├── llm_provider.py
│               ├── semconv.py  └── types.py
│
├── logfire-api/                # Separate workspace member — lightweight no-dep shim
│   ├── logfire_api/            # Stub-only package (mirrors logfire's public API)
│   │   └── __init__.py/.pyi    # No-op implementations + type stubs
│   └── pyproject.toml          # name=logfire-api, version=4.31.0, no dependencies
│
├── tests/                      # Full test suite (~50 test modules)
│   ├── conftest.py             # Shared pytest fixtures
│   ├── otel_integrations/      # Per-framework integration tests
│   ├── exporters/              # Exporter unit tests
│   ├── cassettes/              # VCR HTTP cassette recordings
│   ├── auto_trace_samples/     # Sample code for auto-trace AST tests
│   └── test_*.py               # Unit and integration tests
│
├── docs/                       # MkDocs documentation source
├── examples/                   # Usage examples (python/, javascript/)
├── pyodide_test/               # Browser/Emscripten compatibility test (Node.js)
├── plans/                      # Internal planning documents
├── release/                    # Release tooling
├── .github/
│   ├── workflows/
│   │   ├── main.yml            # Primary CI/CD pipeline
```

```
│   │   ├── weekly_deps_test.yml
│   │   ├── check_changelog.yml
│   │   ├── claude.yml          # Claude AI agent workflow
│   │   └── copilot-setup-steps.yml
│   ├── agents/                 # AI agent configuration
│   └── skills/                 # GitHub Copilot skills
├── pyproject.toml              # Root package manifest (logfire 4.31.0)
├── uv.lock                     # Committed lockfile
└── Makefile                    # Developer convenience targets
```

---

3. Entry Points

Entry Point	Type	File	De
`logfire` (CLI)	CLI	`logfire/cli.py` → `logfire/_internal/cli/__init__.py`	Top-leve point re `pyproje` `[project`
`logfire auth`	CLI subcommand	`logfire/_internal/cli/auth.py`	Device- flow to with Log
`logfire auth logout`	CLI subcommand	`logfire/_internal/cli/auth.py`	Remove tokens
`logfire whoami`	CLI subcommand	`logfire/_internal/cli/__init__.py`	Print cu authent and pro

Entry Point	Type	File	De
`logfire clean`	CLI subcommand	`logfire/_internal/cli/__init__.py`	Remove `.logfir` credenti
`logfire inspect`	CLI subcommand	`logfire/_internal/cli/__init__.py`	Inspect package OTel ins package
`logfire projects list`	CLI subcommand	`logfire/_internal/cli/__init__.py`	List proj authent write ac
`logfire projects new`	CLI subcommand	`logfire/_internal/cli/__init__.py`	Create a project
`logfire projects use`	CLI subcommand	`logfire/_internal/cli/__init__.py`	Select a project credenti
`logfire read-tokens create`	CLI subcommand	`logfire/_internal/cli/__init__.py`	Create a for a pro
`logfire prompt`	CLI subcommand	`logfire/_internal/cli/prompt.py`	Fetch ar prompt issue (C Codex, integrat

Entry Point	Type	File	De
`logfire info`	CLI subcommand	`logfire/_internal/cli/__init__.py`	Print version/ diagnos
`logfire run`	CLI subcommand	`logfire/_internal/cli/run.py`	Run a Py module instrum applied
`python -m logfire`	CLI (module)	`logfire/__main__.py`	Alias for CLI
`logfire.configure()`	Python API	`logfire/_internal/config.py`	Initialize must be tracing
`logfire.span()` / `logfire.instrument()`	Python API	`logfire/_internal/main.py`	Core tra (context decorat
`logfire.{trace,debug,info,notice,warn,error,fatal,exception}()`	Python API	`logfire/_internal/main.py`	Structur emission severity
`logfire.metric_*()`	Python API	`logfire/_internal/main.py`	OTel me (counter gauge, u counter,

Entry Point	Type	File	De
`logfire.install_auto_tracing()`	Python API	`logfire/_internal/main.py`	Install A import h trace m
`logfire.instrument_*()` (×30)	Python API	`logfire/_internal/main.py`	Per-fram instrum wrapper Django, Anthrop
`logfire.variables_*()` / `logfire.var`	Python API	`logfire/__init__.py`	Manage (remote subsyste
`logfire.get_context()` / `logfire.attach_context()`	Python API	`logfire/propagate.py`	W3C TraceCo propaga
`LogfireQueryClient` / `AsyncLogfireQueryClient`	Python API	`logfire/experimental/query_client.py`	SQL que reading telemet read AP
`logfire.db_api.connect()`	Python API	`logfire/db_api.py`	PEP 249 connect the que (pandas Jupyter,

Entry Point	Type	File	De
`LogfireAPIClient`	Python API	`logfire/experimental/api_client.py`	Manage AI evalu Logfire
`forward_export_request()` / `logfire_proxy`	Python API	`logfire/experimental/forwarding.py`	Forward requests WSGI ap
`logfire.testing` (pytest plugin)	Pytest plugin	`logfire/testing.py`	Auto-re plugin (entry pc providir fixture
`logfire._internal.integrations.pytest`	Pytest plugin	`logfire/_internal/integrations/pytest.py`	Seconda plugin e (`pytest`
`logfire.integrations.pydantic:plugin`	Framework plugin	`logfire/integrations/pydantic.py`	Pydantic entry pc instrum automa installec

---

4. Data Stores

Store	Type	Technology	Location / Config
User authentication tokens	File — local	TOML (hand-written)	`~/.logfire/default.toml` — keys are base URLs, values are `token` + `expiration` fields
Project write credentials	File — local	JSON	`.logfire/logfire_credentials.json` — per project working directory; configurable via `LOGFIRE_CREDENTIALS_DIR` / `data_dir`
CLI activity log	File — local	Plain text	`~/.logfire/log.txt` — written by Python `logging.FileHandler` during CLI execution
Failed export retry queue	File — temp	Raw bytes (serialised protobuf OTLP payloads)	OS temp dir under `logfire-retryer-*` prefix (`mkdtemp`); managed by `DiskRetryer` in `logfire/_internal/exporters/otlp.py` ; max 512 MB; daemon thread drains the queue
pyproject.toml config	File — local	TOML	`pyproject.toml` `[tool.logfire]` section in config/working directory; read-only by SDK
Logfire platform (telemetry backend)	Remote — SaaS	OTLP/HTTP + Protobuf	`https://logfire-us.pydantic.dev` (US region) or `https://logfire-eu.pydantic.dev` (EU region); write token in `Authorization` header
In-memory span/log buffers	In-process memory	OTel `BatchSpanProcessor` / `BatchLogRecordProcessor` queues	Runtime only; flushed on `logfire.force_flush()` / `logfire.shutdown()`

Store	Type	Technology	Location / Config
In-memory test exporter	In-process memory	`TestExporter` / `TestLogExporter`	Used in tests and with `CaptureLogfire`; no persistence

---

5. External Integrations

Service	Category	SDK / Client	Purpose
Logfire Platform (US)	Observability backend	`requests.Session` (OTLP HTTP)	Telemetry ingest: `https://logfire-us.pydantic.dev`
Logfire Platform (EU)	Observability backend	`requests.Session` (OTLP HTTP)	Telemetry ingest: `https://logfire-eu.pydantic.dev`
Logfire Management API	SaaS management	`requests.Session` (`LogfireClient`)	Project CRUD, user info, token issuance — `/v1/device-auth/new/`, `/v1/device-auth/wait/{device_code}`, project endpoints
Logfire Read/Query API	Data query	`httpx.Client` / `httpx.AsyncClient`	SQL query against stored telemetry; read token auth; JSON or Apache Arrow response
Logfire Variables API	Remote config	`requests.Session`	Pull managed variable values; background-thread polling with long-poll semantics

Service	Category	SDK / Client	Purpose
OpenAI	AI/LLM	`openai` package (instrumentation wrapper)	Instrument chat completion, embedding, image generation calls as OTel spans
OpenAI Agents SDK	AI/Agentic	`agents` / `openai-agents` package (custom `TraceProvider`)	Instrument multi-agent runs; converts OpenAI Agents traces to OTel spans
Anthropic	AI/LLM	`anthropic` package (instrumentation wrapper)	Instrument Anthropic Claude API calls
Claude Agent SDK	AI/Agentic	`claude-agent-sdk` (hook-based)	Instrument Claude Code / agent SDK runs
Google Generative AI	AI/LLM	`opentelemetry-instrumentation-google-genai`	Instrument Google Gemini API calls
LiteLLM	AI/LLM proxy	`openinference-instrumentation-litellm`	Instrument LiteLLM multi-provider calls
DSPy	AI/LLM	`openinference-instrumentation-dspy`	Instrument DSPy LM programs
Pydantic AI	AI framework	`pydantic-ai-slim` (`InstrumentationSettings`)	Hook into Pydantic AI's native OTel instrumentation
LangChain / LangGraph	AI framework	`langchain`, `langgraph` (dev dep)	Tested as an integration target

Service	Category	SDK / Client	Purpose
MCP (Model Context Protocol)	AI tool protocol	`mcp` package	Instrument MCP client/server sessions (JSON-RPC 2.0 spans)
FastAPI	Web framework	`opentelemetry-instrumentation-fastapi`	HTTP request/response tracing
Django	Web framework	`opentelemetry-instrumentation-django`	HTTP request/response tracing
Flask	Web framework	`opentelemetry-instrumentation-flask`	HTTP request/response tracing
Starlette	Web framework	`opentelemetry-instrumentation-starlette`	HTTP request/response tracing
ASGI (generic)	Web protocol	`opentelemetry-instrumentation-asgi`	Generic ASGI middleware tracing
WSGI (generic)	Web protocol	`opentelemetry-instrumentation-wsgi`	Generic WSGI middleware tracing
httpx	HTTP client	`opentelemetry-instrumentation-httpx`	Outbound async HTTP request tracing
requests	HTTP client	`opentelemetry-instrumentation-requests`	Outbound HTTP request tracing

Service	Category	SDK / Client	Purpose
aiohttp (client)	HTTP client	`opentelemetry-instrumentation-aiohttp-client`	Outbound async HTTP tracing
aiohttp (server)	HTTP server	`opentelemetry-instrumentation-aiohttp-server`	Inbound async HTTP tracing
SQLAlchemy	ORM/DB	`opentelemetry-instrumentation-sqlalchemy`	DB query tracing
asyncpg	PostgreSQL	`opentelemetry-instrumentation-asyncpg`	Async PostgreSQL query tracing
psycopg / psycopg2	PostgreSQL	`opentelemetry-instrumentation-psycopg` / `psycopg2`	PostgreSQL query tracing
SQLite3	SQLite	`opentelemetry-instrumentation-sqlite3`	SQLite query tracing
MySQL	MySQL	`opentelemetry-instrumentation-mysql`	MySQL query tracing
PyMongo	MongoDB	`opentelemetry-instrumentation-pymongo`	MongoDB operation tracing
Redis	Redis	`opentelemetry-instrumentation-redis`	Redis command tracing

Service	Category	SDK / Client	Purpose
Celery	Task queue	`opentelemetry-instrumentation-celery`	Celery task tracing
AWS Lambda	Serverless	`opentelemetry-instrumentation-aws-lambda`	Lambda invocation tracing
SurrealDB	Multi-model DB	`surrealdb` (direct integration)	SurrealDB query tracing
System Metrics	OS/runtime	`opentelemetry-instrumentation-system-metrics`	CPU, memory, network, disk metrics
Algolia	Search / docs analytics	`algoliasearch` (CI / docs build only)	Documentation search index — not in SDK runtime
PyPI	Package registry	N/A	Distribution target for `logfire` and `logfire-api`
stdlib `logging`	Logging bridge	`LogfireLoggingHandler` (internal)	Route Python stdlib log records into OTel
structlog	Logging bridge	`StructlogProcessor` (internal)	Route structlog events into OTel
Loguru	Logging bridge	`LogfireHandler` (internal)	Route loguru log records into OTel

---

6. Authentication & Authorization

- **Auth Model (CLI / Management API)**:
  - **Device Authorization Code Flow** (OAuth 2.0 Device Flow variant): `logfire auth` POSTs to `/v1/device-auth/new/` to obtain a `device_code` + `frontend_auth_url`; the user visits the URL in a browser; the CLI polls `/v1/device-auth/wait/{device_code}` (long-poll, 15-second timeout per request, indefinitely) until the platform issues a `UserTokenData` containing `{token, expiration}`.
  - **User tokens** are persisted in `~/.logfire/default.toml` keyed by base URL. Multiple region tokens can coexist in the same file. Token format: `pylf_v{version}_{region}_{random}` (validated via regex `PYDANTIC_LOGFIRE_TOKEN_PATTERN`).
  - `LogfireClient` sets `Authorization: <user_token>` and `User-Agent: logfire/{version}` on all management API HTTP calls.
- **Auth Model (Telemetry / SDK runtime)**:
  - **Write tokens** (`LOGFIRE_TOKEN` env var or `token=` in `configure()`): Static bearer tokens embedded in OTLP export request `Authorization` HTTP headers. The token encodes the region — the SDK derives the correct backend URL from the token itself via `get_base_url_from_token()`. A list of tokens may be supplied for fan-out to multiple projects.
  - **API keys** (`LOGFIRE_API_KEY` / `api_key=`): Separate credential for the managed-variables (remote config) API.
  - **Read tokens** (issued via `logfire read-tokens create`): Authenticate against the Logfire Query/SQL API (`LogfireQueryClient`).
- **Authorization Pattern**:
  - All API interactions are token-scoped at the project level. Write tokens authorize ingest to a specific project. Read tokens authorize SQL queries against a project's data. User tokens authorize management operations. Authorization is enforced server-side by the closed-source platform — there is no RBAC/policy logic in the SDK.
- **Enforcement Points**:

- `LogfireClient.__init__`: checks token expiry before any API call; raises `RuntimeError` on expired token.
- `UserTokenCollection.get_token()`: raises `LogfireConfigError` if no valid (non-expired) token is found.
- `LogfireConfig` / `configure()`: validates token format and resolves backend URL before any spans are sent.
- OTLP HTTP session: `Authorization` header is set on every export request; backend returns 401/403 on unauthorized.
- Query client: `read_token` sent as a header on each SQL request.

---

7. UI Technology

- **Frontend Framework**: No user interface — backend/library/CLI only.
- **Rendering Model**: N/A. The SDK is a Python library and CLI tool. `rich` is used for styled terminal output.
- **Styling**: `rich` library for terminal colors, tables, progress indicators, and console formatting.

> **Note**: The Logfire web dashboard (observability UI) is **closed source** and hosted separately at `logfire.pydantic.dev`. It is not part of this repository.

---

8. AI/ML Components

The SDK does not implement any AI/ML models itself. It provides **instrumentation wrappers** that capture OpenTelemetry spans, logs, and metrics for calls made to external AI/ML services and frameworks.

Component	Framework / API	Purpose	Location

Component	Framework / API	Purpose	Location
OpenAI instrumentation	`openai` SDK (direct wrapper)	Capture chat completion, embedding, image generation calls as OTel spans; extract prompt/response content, token counts	`logfire/_internal/integrations/llm_providers/openai.py`
Anthropic instrumentation	`anthropic` SDK (direct wrapper)	Capture Anthropic Claude API calls as OTel spans	`logfire/_internal/integrations/llm_providers/anthropic`
LLM provider base	Internal	Shared span attribute naming conventions (GenAI semantic conventions) and helper logic	`logfire/_internal/integrations/llm_providers/llm_provi` `semconv.py`, `types.py`
Google Generative AI instrumentation	`opentelemetry-instrumentation-google-genai`	Instrument Google Gemini API calls	`logfire/_internal/integrations/google_genai.py`
LiteLLM instrumentation	`openinference-instrumentation-litellm`	Instrument LiteLLM (multi-provider LLM router) calls	`logfire/_internal/integrations/litellm.py`

Component	Framework / API	Purpose	Location
DSPy instrumentation	`openinference-instrumentation-dspy`	Instrument DSPy LM programs	`logfire/_internal/integrations/dspy.py`
OpenAI Agents SDK instrumentation	`agents` / `openai-agents` package	Custom `TraceProvider` wrapper converting OpenAI Agents trace data to OTel spans; handles agent, function, generation, handoff, and voice span types	`logfire/_internal/integrations/openai_agents.py`
Pydantic AI instrumentation	`pydantic-ai-slim` (`InstrumentationSettings`)	Hook into Pydantic AI's native OTel instrumentation via tracer/meter/logger provider injection	`logfire/_internal/integrations/pydantic_ai.py`
Claude Agent SDK instrumentation	`claude-agent-sdk`	Instrument Anthropic Claude Code agent runs via hook-based span capture	`logfire/_internal/integrations/claude_agent_sdk.py`

Component	Framework / API	Purpose	Location
MCP instrumentation	`mcp` package	Instrument Model Context Protocol client sessions and server tool calls (JSON-RPC 2.0 spans)	`logfire/_internal/integrations/mcp.py`
AI evals API client	`pydantic-evals` (optional dep)	Programmatic management of AI evaluation datasets and test cases stored in Logfire platform	`logfire/experimental/api_client.py`
AI evaluation annotations	Internal	Mark spans with AI evaluation outcome labels (pass/fail/score)	`logfire/experimental/annotations.py`

---

9. Dependency Ecosystems

Ecosystem	Package Manager	Manifest	Lockfile Present	Direct Runtime Deps	Dev Deps

Ecosystem	Package Manager	Manifest	Lockfile Present	Direct Runtime Deps	Dev Deps
Python (`logfire`)	`uv`	`pyproject.toml` (root)	Yes — `uv.lock`	8 core + ~20 optional groups	~80
Python (`logfire-api`)	`uv`	`logfire-api/pyproject.toml`	Shared `uv.lock`	0 (no runtime deps; shim only)	0
JavaScript (Pyodide test only)	`npm`	`pyodide_test/package.json`	Yes — `pyodide_test/package-lock.json`	Test tooling only; not part of SDK distribution	N/A

Core runtime dependencies (`logfire` package):

Package	Version Constraint	Role
`opentelemetry-sdk`	`>=1.39.0, <1.40.0`	OTel traces, metrics, logs SDK
`opentelemetry-exporter-otlp-proto-http`	`>=1.39.0, <1.40.0`	OTLP/HTTP protobuf exporter
`opentelemetry-instrumentation`	`>=0.41b0`	Base instrumentation framework
`rich`	`>=13.4.2`	Terminal output / console exporter
`protobuf`	`>=4.23.4`	Protobuf serialization for OTLP
`typing-extensions`	`>=4.1.0`	Backports of typing features
`tomli`	`>=2.0.1; python_version < '3.11'`	TOML config file parsing (Python < 3.11)
`executing`	`>=2.0.1`	Source code inspection for f-string magic

> **Note**: OTel version is **pinned to a narrow minor range** (`1.39.x`) to ensure API stability. This is intentional SDK design but creates a hard compatibility constraint for applications with conflicting OTel version requirements.

---

10. Deployment & Infrastructure

- **Containerization**: None detected. No Dockerfile, docker-compose, or Kubernetes manifests are present. The SDK is distributed as a Python package only.
- **CI/CD**: GitHub Actions
  - `.github/workflows/main.yml` — Primary pipeline: lint (`ruff`), typecheck (`pyright`), docs build (`mkdocs`), test matrix (Python 3.9–3.14 × multiple pydantic/otel versions), PyPI publish on version tag
  - `.github/workflows/weekly_deps_test.yml` — Weekly scheduled test run against latest dependency versions
  - `.github/workflows/check_changelog.yml` — Enforce changelog updates on PRs
  - `.github/workflows/claude.yml` — AI-assisted issue/PR handling via Claude
  - `.github/workflows/copilot-setup-steps.yml` — GitHub Copilot workspace bootstrap
  - Uses **Depot** 4-core Ubuntu runners for maintainer PRs (faster CI); standard GitHub runners for forks
- **Cloud Provider**: GCP (Google Cloud Platform) — `gcp_region` values `us-east4` (US) and `europe-west4` (EU) in `logfire/_internal/auth.py` `REGIONS` dict. This is the deployment region of the **Logfire platform** (closed source); no GCP SDK usage in this repository.
- **IaC**: None detected.
- **Environment Configuration**: All SDK parameters have corresponding `LOGFIRE_*` environment variables (authoritative list in `logfire/_internal/config_params.py`):

Env Var	Purpose
`LOGFIRE_TOKEN`	Write token(s) — comma-separated for multi-project fan-out
`LOGFIRE_API_KEY`	API key for managed variables
`LOGFIRE_SEND_TO_LOGFIRE`	Enable/disable telemetry sending (`true` / `false` / `if-token-present`)
`LOGFIRE_BASE_URL`	Override backend URL (self-hosted / alternative backends)
`LOGFIRE_SERVICE_NAME`	OTel service name
`LOGFIRE_SERVICE_VERSION`	OTel service version
`LOGFIRE_ENVIRONMENT`	Deployment environment (e.g., `prod` , `staging`)
`LOGFIRE_CREDENTIALS_DIR`	Override `.logfire/` credentials directory
`LOGFIRE_CONSOLE`	Enable/disable console output
`LOGFIRE_CONSOLE_*`	Various console display options
`LOGFIRE_MIN_LEVEL`	Minimum telemetry level filter
`LOGFIRE_TRACE_SAMPLE_RATE`	Head sampling rate (0.0–1.0)
`LOGFIRE_INSPECT_ARGUMENTS`	Enable f-string magic
`LOGFIRE_DISTRIBUTED_TRACING`	Whether to accept/extract incoming W3C trace context

Env Var	Purpose
`OTEL_SERVICE_NAME`, `OTEL_EXPORTER_OTLP_ENDPOINT`, `OTEL_TRACES_EXPORTER`, etc.	Standard OTel env vars also respected

File-based config: `[tool.logfire]` section in `pyproject.toml` (loaded from `config_dir`, defaults to CWD).

- **Distribution**: PyPI packages `logfire` and `logfire-api` built with Hatchling. Release tooling in `release/` directory.
- **Pyodide / Browser**: Experimental WebAssembly/browser support — `platform_is_emscripten()` checks in exporters and async utilities disable disk retry and threading in Emscripten context. Tested via `pyodide_test/`.

---

11. Key Observations for Auditors

- **Two-package workspace**: `logfire` (full SDK, MIT-licensed) and `logfire-api` (zero-dependency shim mirroring the public API with no-op stubs, also MIT). The shim lets library authors optionally support Logfire without making it a hard dependency. Both share the same version number (`4.31.0`).
- **Closed-source backend**: The server, dashboard, and data storage infrastructure are **not in this repository**. All audit findings related to server-side data processing, storage, access control, and privacy must be directed at the platform team separately.
- **Sensitive data scrubbing is built-in**: `logfire/_internal/scrubbing.py` applies pattern-based redaction (defaults: `password`, `secret`, `api_key`, `auth`, `session`, `cookie`, `credit_card`, `jwt`, `ssn`, `logfire_token`, `csrf`, `xsrf`) to all span attributes and log record values **before** export. Scrubbing can be disabled (`scrubbing=False`) or customised. Auditors should verify default patterns cover data classification requirements.
- **Write token embedded in every OTLP export request**: The `Authorization` header containing the write token is sent with every OTLP HTTP POST. Leaking this token allows an adversary to ingest arbitrary data into the associated Logfire project. Token format

encodes region and version: `pylf_v{version}_{region}_{token}`.

- **Disk retry with up to 512 MB of application payload**: Failed OTLP exports are serialised (as raw protobuf bytes) to disk in an OS temp directory under a `logfire-retryer-*` prefix (`DiskRetryer`). Payloads are post-scrubbing but may still contain application telemetry data. Files persist until retried or process exits. The directory is not explicitly deleted on shutdown.
- **AST-rewriting auto-trace**: `install_auto_tracing()` installs a `sys.meta_path` import hook that rewrites Python ASTs at import time to inject `with logfire.span():` into every function. This is a powerful and unusual capability that affects any module imported after the hook is installed. The `@no_auto_trace` decorator and `AutoTraceModule` filter type allow exclusion.
- **F-string magic / argument inspection**: `logfire_format()` uses the `executing` library to inspect the calling frame's AST and extract f-string components, enabling rich structured logging from plain Python f-strings. This is enabled by default on Python ≥ 3.11 and relies on CPython implementation details.
- **Multi-token fan-out**: `LOGFIRE_TOKEN` accepts comma-separated tokens; `token=` parameter accepts a `list[str]`. The SDK exports telemetry to all configured projects simultaneously. Useful for migration scenarios but increases network overhead proportionally.
- **Distributed tracing / context injection behaviour**: By default, the SDK warns (but does not block) when incoming W3C `traceparent` headers are detected (`LOGFIRE_DISTRIBUTED_TRACING` unset ⟹ `WarnOnExtractTraceContextPropagator`). Setting to `True` silently accepts external trace context. This is a security-relevant behaviour determining whether untrusted upstream trace IDs are honoured in the instrumented application.
- **pytest integration suppresses telemetry by default**: When the Logfire pytest plugin is active, `send_to_logfire` defaults to `False` (via detection of `PYTEST_VERSION` env var) preventing accidental telemetry sends from test runs. The `capfire` fixture provides in-memory span capture.
- **Pydantic plugin activates at Pydantic import time**: Registered as a `[project.entry-points."pydantic"]` plugin, this hook fires whenever `pydantic` is imported — potentially before any `logfire.configure()` call. If `logfire` is installed in an environment, Pydantic validation activity may be instrumented globally without explicit opt-in.

- **Device code auth — indefinite polling loop**: `poll_for_token()` polls indefinitely (long-poll, 15 s timeout per request). There is no wall-clock abort timeout — only 4 consecutive network errors stop the loop. This is standard for device-flow UX but is worth noting for automated environments.
- **OTel version strictly pinned to minor range**: The `opentelemetry-sdk` and OTLP exporter are pinned to `>=1.39.0, <1.40.0`. Downstream applications requiring a different OTel minor version will have a hard dependency conflict. This is intentional for API stability.
- **Pyodide / browser deployment path**: `platform_is_emscripten()` checks disable threading and disk I/O. This is an unusual deployment target for a Python telemetry SDK and may have different security and privacy characteristics from server-side deployment.
- **No Dockerfile or container configuration**: Runtime security posture depends entirely on the host application's deployment environment, not anything defined in this repository.
- **`logfire run` wraps arbitrary scripts**: The `logfire run <script>` CLI subcommand executes arbitrary Python scripts/modules after installing auto-instrumentation. This is a legitimate developer tool but represents a code-execution surface via the CLI.

Summary

Attribute	Value
Project type	Open-source Python SDK library + CLI tool (OpenTelemetry distro / observability)
Language count	2 (Python primary; JavaScript for Pyodide test harness only)
Package count	2 (`logfire` , `logfire-api`)

Attribute	Value
CLI subcommand count	13 (under the `logfire` command)
Public Python API symbols	~60 exported names in `logfire.__init__`
Data store count	5 (2 local credential files, 1 CLI log file, 1 disk-backed retry queue, 1 remote SaaS backend)
External integration count	~35 (Logfire platform APIs + 30+ framework/service instrumentation targets)
AI/ML components	Yes — instrumentation wrappers only (no embedded models); 10 AI/ML framework integration targets; experimental evals API client and annotation helpers
UI present	No (backend/library/CLI only; Logfire web dashboard is a separate closed-source product)